

Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский университет ИТМО

Факультет программной инженерии и компьютерной техники
Направление подготовки 09.03.04 Программная инженерия
Дисциплина “Сервис-ориентированная архитектура”

Отчёт

По лабораторной работе №1

Вариант: 67615

Студенты:

Таджединов Р. Э.

группа *P3408*

Шмунк А. А.

группа *P3408*

Преподаватель:

Кривоносов Е. Д.

г. Санкт-Петербург, 2026 г.

Задание (вариант 67615)

Внимание! У разных вариантов разный текст задания!

Разработать спецификацию в формате OpenAPI для набора веб-сервисов, реализующего следующую функциональность:

Первый веб-сервис должен осуществлять управление коллекцией объектов. В коллекции необходимо хранить объекты класса `Movie`, описание которого приведено ниже:

```
public class Movie {
    private int id; //Значение поля должно быть больше 0, Значение этого
    поля должно быть уникальным, Значение этого поля должно
    генерироваться автоматически
    private String name; //Поле не может быть null, Строка не может быть
    пустой
    private Coordinates coordinates; //Поле не может быть null
    private java.time.LocalDate creationDate; //Поле не может быть null,
    Значение этого поля должно генерироваться автоматически
    private int oscarsCount; //Значение поля должно быть больше 0
    private MovieGenre genre; //Поле может быть null
    private Mpaarating mpaarating; //Поле не может быть null
    private Person screenwriter;
}

public class Coordinates {
    private int x;
    private long y;
}

public class Person {
    private String name; //Поле не может быть null, Строка не может быть
    пустой
    private String passportID; //Длина строки должна быть не меньше 5, Поле
    может быть null
    private Color hairColor; //Поле может быть null
    private Country nationality; //Поле не может быть null
}

public enum MovieGenre {
```

```
ACTION,  
DRAMA,  
COMEDY,  
MUSICAL,  
HORROR;  
}  
  
public enum MpaaRating {  
    G,  
    PG,  
    PG_13,  
    R,  
    NC_17;  
}  
  
public enum Color {  
    RED,  
    BLUE,  
    ORANGE,  
    WHITE;  
}  
  
public enum Country {  
    RUSSIA,  
    UNITED_KINGDOM,  
    FRANCE,  
    SPAIN,  
    NORTH_KOREA;  
}
```

Веб-сервис должен удовлетворять следующим требованиям:

- API, реализуемый сервисом, должен соответствовать рекомендациям подхода RESTful.
- Необходимо реализовать следующий базовый набор операций с объектами коллекции: добавление нового элемента, получение элемента по ИД, обновление элемента, удаление элемента, получение массива элементов.
- Операция, выполняемая над объектом коллекции, должна определяться методом HTTP-запроса.
- Операция получения массива элементов должна поддерживать возможность сортировки и фильтрации по любой комбинации полей класса, а также возможность постраничного вывода результатов

выборки с указанием размера и порядкового номера выводимой страницы.

- Все параметры, необходимые для выполнения операции, должны передаваться в URL запроса.
- Информация об объектах коллекции должна передаваться в формате **json**.
- В случае передачи сервису данных, нарушающих заданные на уровне класса ограничения целостности, сервис должен возвращать код ответа `http`, соответствующий произошедшей ошибке.

Помимо базового набора, веб-сервис должен поддерживать следующие операции над объектами коллекции:

- Удалить все объекты, значение поля `genre` которого эквивалентно заданному.
- Вернуть один (любой) объект, значение поля `name` которого является минимальным.
- Вернуть массив объектов, значение поля `name` которых начинается с заданной подстроки.

Эти операции должны размещаться на отдельных URL.

Второй веб-сервис должен располагаться на URL `/oscar`, и реализовывать ряд дополнительных операций, связанных с вызовом API первого сервиса:

- `/movies/reward-r` : добавить всем фильмам категории "R" по одному "Оскару"
- `/genres/redistribute-rewards/{from-genre}/{to-genre}` : равномерно перераспределить все "Оскары", полученные фильмами одного жанра, между фильмами другого жанра

Эти операции также должны размещаться на отдельных URL.

Для разработанной спецификации необходимо сгенерировать интерактивную веб-документацию с помощью Swagger UI. Документация должна содержать описание всех REST API обоих сервисов с текстовым описанием функциональности каждой операции. Созданную веб-документацию необходимо развернуть на сервере `helios`.

Задания к лабораторным работам

(COA) Лабораторная работа №1

Введите вариант:

67615

Внимание! У разных вариантов разный текст задания!

Разработать спецификацию в формате OpenAPI для набора веб-сервисов, реализующего следующую функциональность:

Первый веб-сервис должен осуществлять управление коллекцией объектов. В коллекции необходимо хранить объекты класса `Movie`, описание которого приведено ниже:

```
public class Movie {
    private int id; //Значение поля должно быть больше 0, Значение этого поля должно быть уникальным, Значение этого поля должно генерироваться автоматически
    private String name; //Поле не может быть null, Строка не может быть пустой
    private Coordinates coordinates; //Поле не может быть null
    private java.time.LocalDate creationDate; //Поле не может быть null, Значение этого поля должно генерироваться автоматически
    private int oscarCount; //Значение поля должно быть больше 0
    private MovieGenre genre; //Поле может быть null
    private Mpaarating mpaarating; //Поле не может быть null
    private Person screenwriter;
}

public class Coordinates {
    private int x;
    private long y;
}

public class Person {
    private String name; //Поле не может быть null, Строка не может быть пустой
    private String passportID; //Длина строки должна быть не меньше 5, Поле может быть null
    private Color hairColor; //Поле может быть null
    private Country nationality; //Поле не может быть null
}

public enum MovieGenre {
    ACTION,
    DRAMA,
    COMEDY,
    MUSICAL,
    HORROR;
}

public enum Mpaarating {
    G,
    PG,
    PG_13,
    R,
    NC_17;
}

public enum Color {
    RED,
    BLUE,
    ORANGE,
    WHITE;
}

public enum Country {
    RUSSIA,
    UNITED_KINGDOM,
    FRANCE,
    SPAIN,
    NORTH_KOREA;
}
```

Веб-сервис должен удовлетворять следующим требованиям:

- API, реализуемый сервисом, должен соответствовать рекомендациям подхода RESTful.
- Необходимо реализовать следующий базовый набор операций с объектами коллекции: добавление нового элемента, получение элемента по ID, обновление элемента, удаление элемента, получение массива элементов.
- Операция, выполняемая над объектом коллекции, должна определяться методом HTTP-запроса.
- Операция получения массива элементов должна поддерживать возможность сортировки и фильтрации по любой комбинации полей класса, а также возможность страничного вывода результатов выборки с указанием размера и порядкового номера выводимой страницы.
- Все параметры, необходимые для выполнения операции, должны передаваться в URL запроса.
- Информация об объектах коллекции должна передаваться в формате json.
- В случае передачи сервису данных, нарушающих заданные на уровне класса ограничения целостности, сервис должен возвращать код ответа http, соответствующий произошедшей ошибке.

Помимо базового набора, веб-сервис должен поддерживать следующие операции над объектами коллекции:

- Удалить все объекты, значение поля `genre` которого эквивалентно заданному.
- Вернуть один (любой) объект, значение поля `name` которого является минимальным.
- Вернуть массив объектов, значение поля `name` которых начинается с заданной подстроки.

Эти операции должны размещаться на отдельных URL.

Второй веб-сервис должен располагаться на URL `/oscar`, и реализовывать ряд дополнительных операций, связанных с вызовом API первого сервиса:

- `/movies/reward-r` : добавить всем фильмам категории "R" по одному "Оскару"
- `/genres/redistribute-rewards/{from-genre}/{to-genre}` : равномерно перераспределить все "Оскары", полученные фильмами одного жанра, между фильмами другого жанра

Эти операции также должны размещаться на отдельных URL.

Для разработанной спецификации необходимо сгенерировать интерактивную веб-документацию с помощью Swagger UI. Документация должна содержать описание всех REST API обоих сервисов с текстовым описанием функциональности каждой операции. Созданную веб-документацию необходимо развернуть на сервере `helios`.

</> Вопросы к защите лабораторной работы №1

1. Подходы к проектированию приложений. "Монолитная" и сервис-ориентированная архитектура.
 2. Понятие сервиса. Общие свойства сервисов.
 3. Основные принципы SOA. Подходы к реализации SOA, стандарты и протоколы.
 4. Общие принципы построения и элементы сервис-ориентированных систем.
 5. Понятие веб-сервиса. Определение, особенности, отличия от веб-приложений.
 6. Категоризация веб-сервисов. RESTful и SOAP. Сходства и отличия, области применения.
 7. RESTful веб-сервисы. Особенности подхода. Понятия ресурса, URI и полезной нагрузки (payload).
 8. Виды RESTful-сервисов. Интерпретация методов HTTP в RESTful.
 9. Правила именования ресурсов в RESTful сервисах.
 10. Спецификация RESTful-сервисов. Стандарт OpenAPI.
 11. Автодокументирование RESTful-сервисов. Swagger Editor, Swagger UI (и Swagger Codegen).
 12. Архитектурный принцип HATEOAS.
-

Исходный код

Репозиторий: <https://github.com/r4m63/soa-itmo>

OpenAPI (docs/openapi.yaml) <https://github.com/r4m63/soa-itmo/blob/main/docs/openapi.yaml>

Swagger, развёрнутый на helios: <https://se.ifmo.ru/~s409659/swagger/>

Спецификация написана в OpenAPI и описывает 2 веб-сервиса. Movies service управляет коллекцией объектов Movie. Oscar service размещён под префиксом /oscar и выполняет дополнительные операции, обращаясь к API первого сервиса.

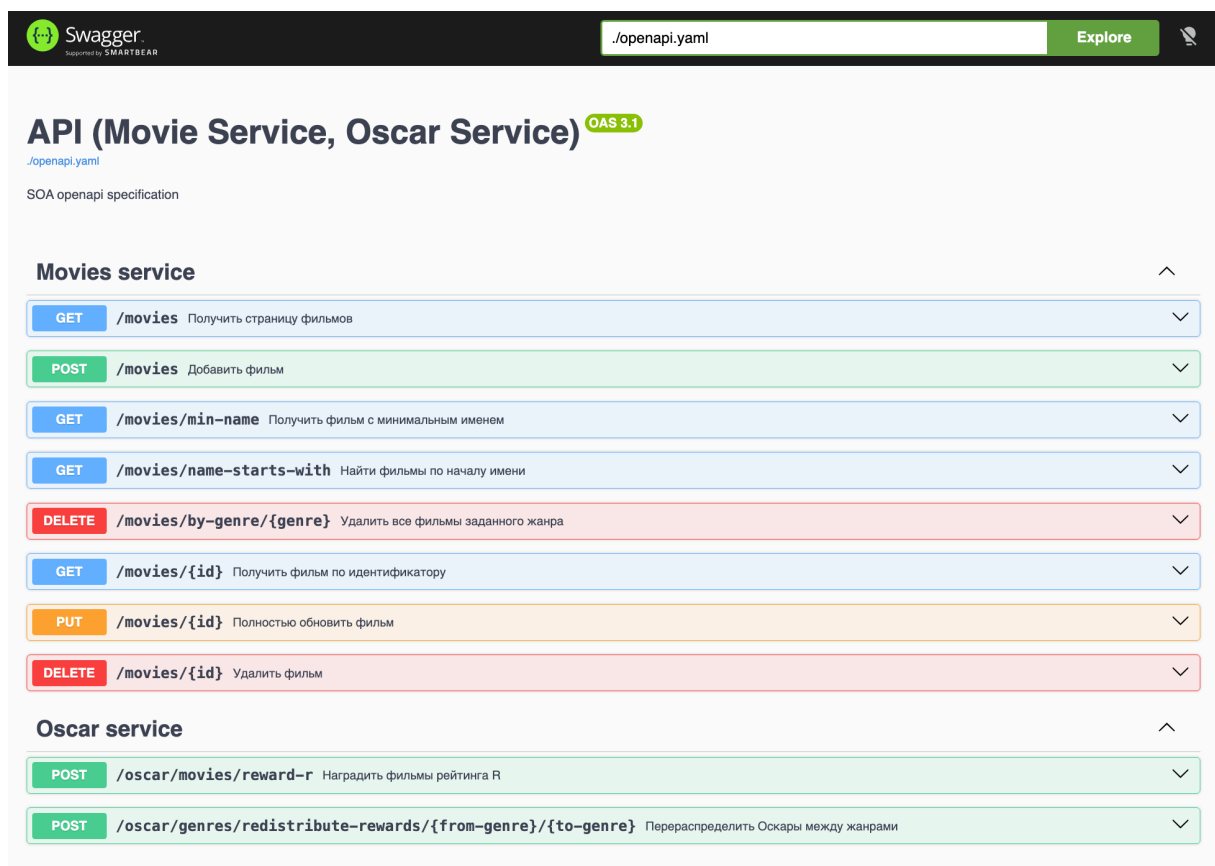


Рисунок 1 — Swagger UI (<https://se.ifmo.ru/~s409659/swagger/>)

Инструкция по развёртыванию

Результат работы -- статическая веб-спецификация. На helios каталог ~/public_html пользователя раздаётся apache веб сервером по адресу https://se.ifmo.ru/~<freebsd_username>/swagger. Достаточно положить туда дистрибутив Swagger UI вместе со спецификацией.

Вывод

Мы спроектировали API двух взаимодействующих сервисов.

Научились писать руками `openapi.yml`